SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission

Student Name	Mukesh Tharun S
Register Number	1971260006
Course	B.Tech Computer Science and Bio Science
Course Code & Name	C Programming
Slot	D
Assignment / Task Title	CO1-Assignment-3: Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative
GitHub Repository URL	https://github.com/mukeshtharun/CSA02-1971260006D-Assignment1
Submission Date	

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

Question Q1: Overview

Question Q1 of CO1-Assignment-3 asks for a billing program for **Grama Jyothi Rural Electricity Co-operative**. The co-operative bills about 4,000 households every month. Its existing spreadsheet produced some bills that were a few paise wrong, gave negative bills when a meter rolled over from 65535 to 0, and typed the tariff in many places, so a tariff revision was applied only partly. The program must calculate every bill correctly to the paisa and must scale from one village (4,000 houses) to one taluk (60,000) to one district (800,000).

The question is divided into four tasks, worth 4, 4, 3 and 4 marks. Task 1 is analysis. Tasks 2, 3 and 4 each have a C program to complete and run. A Design Challenge on smart meters is answered after Task 4.

The lab requires the code, input files and output screenshots to be kept in a GitHub repository named CSA02-<RegisterNumber>-Assignment1, with one folder per task and a README.md. Every program in this report was compiled with gcc 13.3.0 on Ubuntu 24.04 using gcc -std=c11 -Wall -Wextra -pedantic, which produced no warnings. The output shown after each program is the actual output from that run. Each figure is a terminal capture of the real compile-and-run command written in its caption, made in the build environment. If your lab requires screenshots taken on your own machine, replace the figures with your own captures.

TASK 1: Understand the Problem and Identify the Data

Task / Problem Statement

Analyse the billing problem and identify every item of data the program must store and compute. Discuss the inputs (previous reading, current reading, household number, category) and the outputs (units, energy charge, fixed charge, total). Identify which values never change during a run and should be constants, which values can be negative, fractional or very large, and the rules every correct bill must satisfy. State any assumptions and justify each data item.

Aim

To identify every input, output, constant, rule and assumption in the billing problem before any code is written.

Concept / Theory

Good program design starts with the data. Each value is either an **input** (it comes from outside the program, such as a meter reading), a **computed output** (it is calculated, such as the units used), or a **constant** (it never changes during a run, such as the fixed charge). Some rules must always hold. For example, a bill is never negative, and the total is always the fixed charge plus the energy charge. If these rules are written down first, errors such as negative bills can be caught by design.

Procedure / Algorithm

1. Read the scenario and list every value that the meter reader or the office handles.
2. Classify each value as an input, a computed output or a constant.
3. For each value, decide the range it can take: whether it can be negative, fractional or very large.
4. Write the rules that a correct bill must satisfy.
5. Write down the assumptions needed to fill any gaps in the scenario.

Program / Source Code

No program is required for Task 1. The lab asks for analysis only, so this section contains no code.

Output Screenshot

Not applicable. Task 1 has no program to run.

Analysis / Discussion

Data items

Data item	Kind	Range or size	Justification
Household number	Input	1 to 800,000	Identifies the bill. It is never used in arithmetic, so it is an integer.

Data item	Kind	Range or size	Justification
Category	Input	One letter code	Identifies the consumer type. The scenario gives no category-specific rates.
Previous reading	Input	0 to 65,535	The meter register is a 16-bit counter, and units are whole numbers.
Current reading	Input	0 to 65,535	Same as above. It can be smaller than the previous reading after a rollover.
Units consumed	Computed	0 to 65,535	Never negative. A rollover must be handled by wrap-around arithmetic.
Energy charge	Computed	0 to about 29.4 million paise	Depends on the slabs. Must be exact to the paisa.
Fixed charge	Constant	5,000 paise (Rs 50)	Charged on every bill, even when units are zero.
Total bill	Computed	Paise, never negative	Equals the fixed charge plus the energy charge.
Tariff rates and slab limits	Constants	Rates 0, 225 and 450 paise; limits 100 and 200 units	Must be defined once so that a revision changes every bill.
District total	Computed	About 2.4 × 10^10 paise	Sum of all bills. Assumes an average bill of Rs 300 for 800,000 houses. This value is too large for a 32-bit int.

The largest possible energy charge is calculated as follows. The first 100 units are free. The next 100 units cost 100 × 225 = 22,500 paise. The remaining 65,335 units cost 65,335 × 450 = 29,400,750 paise. The total is 29,423,250 paise, which is about Rs 294,232.50. Fractional rupee values such as Rs 2.25 per unit are therefore stored as whole paise.

Rules a correct bill must satisfy

- Units consumed is zero or more, including after a meter rollover.
- A bill is never negative. The smallest possible bill is the fixed charge of Rs 50.
- The total bill equals the fixed charge plus the energy charge, exactly.
- Each unit is charged in exactly one slab. No unit is counted twice and no unit is missed.
- The same units always give the same bill.

Assumptions

- The meter rolls over at most once in a billing period, so consumption is less than 65,536 units.
- Readings are whole units. There are no decimal readings.
- Only the fixed charge and the slab rates in the scenario apply. There are no taxes, surcharges or rebates.
- The fixed charge is charged even when consumption is zero.
- The household category is stored for identification. It does not change the calculation.
- Smart meter data is not considered here. That is the subject of the Design Challenge.

Alternative concept and complexity analysis: readings stored as text

If each reading were stored as text, such as "01530", the solution would need a string of five digits plus the terminating NUL character. That is 6 bytes per reading, compared with 2 bytes for an unsigned short. Converting a string to a number takes time proportional to the number of digits d, so each conversion is O(d). Comparing two readings as text is also O(d) in the worst case, while comparing two integers is O(1). For 800,000 houses with two readings each, the text form needs about 9.6 MB (1.6 million readings × 6 bytes), and the binary form needs about 3.2 MB (1.6 million × 2 bytes). Because the register always has exactly five digits, the time difference is only a constant factor. Integers are still preferable because they are simpler and avoid bugs with leading zeros.

Result

Task 1 is complete. All data items, constants, rules and assumptions were identified and justified. They are the basis for the type choices in Task 2.

TASK 2: Choose the Data Types

Task / Problem Statement

Select a C data type for every data item identified in Task 1. For each type, state its size using sizeof and its range using limits.h and float.h, and decide whether it is sufficient for a district. Explain why money should not be stored as float rupees, and what happens to a 16-bit unsigned meter register after 65535. Complete TODO 1 of the consumption() function without using if-else, and check the output against the expected output given in the lab.

Aim

To choose correct C data types for the billing data, to measure their sizes and ranges on this system, and to show that the consumption calculation works across a meter rollover.

Concept / Theory

C provides integer types (char, short, int, long and long long). Each has a minimum size set by the C standard and a range given in limits.h. Unsigned types store only zero and positive values, and they wrap around when they overflow. For an unsigned 16-bit value, the maximum is 65535, and adding 1 gives 0. Floating-point types (float and double) store values in binary, so many decimal fractions, such as 27.30, cannot be stored exactly. A small error on each bill becomes a large error over many bills. Storing money as an integer number of paise avoids this problem completely.

- The result of an unsigned operation is always reduced modulo 65536 for a 16-bit type. So (current − previous) modulo 65536 gives the correct consumption when the register has rolled over once.
- float stores about 6 significant decimal digits (FLT_DIG = 6). double stores about 15 (DBL_DIG = 15).

Procedure / Algorithm

1. Declare meter_t as unsigned short, which is 2 bytes and holds 0 to 65535, matching the 16-bit meter register.
2. Compute the difference current − previous. The difference can be negative, so it is computed as a signed int.
3. Convert the difference to meter_t. This conversion reduces the value modulo 65536. For example, −65280 becomes 256.
4. Return the result as unsigned int, so the function never returns a negative number.
5. Test with a normal reading (1200 to 1530, expected 330) and with a rollover (65400 to 120, expected 256).

Program / Source Code

Program 1: consumption() with TODO 1 completed. The rest of the lab code is unchanged.

```
#include <stdio.h>
#include <limits.h>
/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */
```

```
typedef unsigned short meter_t;
unsigned int consumption(meter_t previous, meter_t current)

{
    // TODO 1: Return the units consumed, correct even when the register has
rolled over
    //   (current < previous); use unsigned 16-bit arithmetic instead of an if-
else
    return (meter_t)(current - previous);
}
int main(void)

{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));   /* rolled over
*/
    return 0;
}
```

Program 2: type sizes and ranges on this system. This program uses sizeof, limits.h and float.h to measure the data types.

```
#include <stdio.h>
#include <limits.h>
#include <float.h>
int main(void){
  printf("char      %zu  %d .. %d\n", sizeof(char), CHAR_MIN, CHAR_MAX);
  printf("short     %zu  %d .. %d\n", sizeof(short), SHRT_MIN, SHRT_MAX);
  printf("int       %zu  %d .. %d\n", sizeof(int), INT_MIN, INT_MAX);
  printf("long      %zu  %ld .. %ld\n", sizeof(long), LONG_MIN, LONG_MAX);
  printf("long long %zu  %lld .. %lld\n", sizeof(long long), LLONG_MIN,
LLONG_MAX);
  printf("unsigned int %zu  0 .. %u\n", sizeof(unsigned int), UINT_MAX);
  printf("float     %zu  max %e  digits %d\n", sizeof(float), FLT_MAX, FLT_DIG);
  printf("double    %zu  max %e  digits %d\n", sizeof(double), DBL_MAX, DBL_DIG);
  return 0;}
```

Program 3: float versus paise. This supporting program adds 4,000 bills of Rs 27.30 using float, double and integer paise, to show the rounding problem.

```
#include <stdio.h>

int main(void)
{
    float  total_f = 0.0f;   /* money as float rupees      */
    double total_d = 0.0;    /* money as double rupees     */
    long   total_p = 0;      /* money as paise (integer)   */
    int i;

    for (i = 0; i < 4000; i++) {
        total_f += 27.30f;
        total_d += 27.30;
        total_p += 2730L;    /* Rs 27.30 = 2730 paise      */
    }
```

```
    printf("Exact total (4000 bills of Rs 27.30): Rs 109200.00\n");
    printf("float  total: Rs %.2f\n", total_f);
    printf("double total: Rs %.2f\n", total_d);
    printf("paise  total: Rs %ld.%02ld\n", total_p / 100, total_p % 100);
    return 0;
}
```

Output Screenshot

```
Task 2, Program 1: consumption.c

$ gcc -std=c11 -Wall -Wextra -pedantic task2/consumption.c -o task2/consumption &&
  ./task2/consumption
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

Figure 2.1: Compiling and running consumption.c

Text copy of the Program 1 output:

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

```
Task 2, Program 2: type sizes and ranges

$ gcc -std=c11 -Wall task2/types.c -o task2/types && ./task2/types
char      1  -128 .. 127
short     2  -32768 .. 32767
int       4  -2147483648 .. 2147483647
long      8  -9223372036854775808 .. 9223372036854775807
long long 8  -9223372036854775808 .. 9223372036854775807
unsigned int 4  0 .. 4294967295
float     4  max 3.402823e+38  digits 6
double    8  max 1.797693e+308  digits 15
```

Figure 2.2: Compiling and running the type-size program

Text copy of the Program 2 output:

```
char      1  -128 .. 127
short     2  -32768 .. 32767
int       4  -2147483648 .. 2147483647
long      8  -9223372036854775808 .. 9223372036854775807
long long 8  -9223372036854775808 .. 9223372036854775807
unsigned int 4  0 .. 4294967295
float     4  max 3.402823e+38  digits 6
double    8  max 1.797693e+308  digits 15
```

```
Task 2, Program 3: float versus paise

$ gcc -std=c11 -Wall -Wextra -pedantic task2_float/float_vs_paise.c -o
  task2_float/float_vs_paise && ./task2_float/float_vs_paise
Exact total (4000 bills of Rs 27.30): Rs 109200.00
float  total: Rs 109196.33
double total: Rs 109200.00
paise  total: Rs 109200.00
```

Figure 2.3: Compiling and running the float-versus-paise program

Text copy of the Program 3 output:

```
Exact total (4000 bills of Rs 27.30): Rs 109200.00
float  total: Rs 109196.33
double total: Rs 109200.00
paise  total: Rs 109200.00
```

Analysis / Discussion

Data type table

Data item	Type chosen	Size (bytes)	Range on this system	Sufficient?
House number	unsigned int	4	0 to 4,294,967,295	Yes. 800,000 is far below the limit.
Category	char	1	−128 to 127	Yes. A single letter code fits.
Meter reading	unsigned short (meter_t)	2	0 to 65,535	Yes. It matches the 16-bit register.
Units consumed	unsigned int	4	0 to 4,294,967,295	Yes.
Energy, fixed charge and bill (paise)	long	8	±9.22 × 10^18	Yes. The C standard guarantees 32 bits, and one bill needs at most about 29.4 million paise.
District total (paise)	long long	8	±9.22 × 10^18	Yes. Needed for portability. The sum of 800,000 bills (about 2.4 × 10^10 paise) exceeds the int limit of 2,147,483,647.
Tariff rates and slab limits	int (macros in Task 3)	4	±2,147,483,647	Yes.

Why money should not be stored as float rupees

Program 3 adds 4,000 bills of Rs 27.30. The exact total is Rs 109,200.00. The float total printed was Rs 109,196.33, which is wrong by Rs 3.67. The value 27.30 cannot be stored exactly in binary, and every addition rounds again, so the errors accumulate. The double total was correct in this run, but the same rounding rules still apply, so double cannot be trusted for exact money. The integer paise total was exact, because 2,730 paise is a whole number and integer addition is exact until overflow. This is the reason the billing program stores all money in paise.

What happens to the 16-bit register after 65535

The 16-bit register counts from 0 to 65535. After 65535, the next reading is 0. In the example, the previous reading is 65400 and the current reading is 120. The units used are 65536 − 65400 + 120 = 256. If plain signed subtraction were used, 120 − 65400 would give −65280, which is a negative number and would produce a negative bill. Storing the difference in an unsigned 16-bit type reduces it modulo 65536, so the result is 256 and is always positive.

Alternative concept and complexity analysis: money stored as double rupees

If money were stored as double rupees, the program would add values such as 162.50 directly. Each addition takes O(1) time, and a double takes 8 bytes, the same as a long. Rounding would have to be applied to each bill, for example with round() or with printf formatting, before it is added to a total. The rounding error of each addition is small, but it accumulates: in the worst case, the error after n additions grows roughly in proportion to n. Over millions of bills, this becomes visible. Paise stored as integers have no rounding at all, so nothing needs to be rounded after a bill is calculated.

Result

Task 2 is complete. The data types are chosen and justified with their sizes and ranges. TODO 1 is implemented without if-else. The program prints 330 units for the normal reading and 256 units for the rollover, which matches the expected output. The supporting program shows that float gives a wrong total for 4,000 bills, while integer paise gives the correct total.

TASK 3: Define the Constants

Task / Problem Statement

Define the tariff values and limits as constants so that a tariff revision is made in one place only. Complete TODO 1 in compute_bill() using only the macros defined at the top, so that the energy charge is calculated slab by slab. The expected outputs are: 0 units → Rs 50.00; 100 units → Rs 50.00; 150 units → Rs 162.50; 200 units → Rs 275.00; 350 units → Rs 950.00. Then change RATE2_PAISE to 230 and check that every bill changes consistently.

Aim

To keep all tariff values in one place using #define macros, and to compute a correct slab-based bill in paise.

Concept / Theory

- **#define** is a preprocessor directive. Before compilation, the preprocessor replaces every use of the macro name with its value. A #define has no type, uses no memory at run time, and is usually not visible in the debugger.
- **const variable** has a type, a scope and an address. The compiler checks its type, the debugger can show it, and it may occupy memory. In C, a const variable is not a constant expression, so it cannot be used for array sizes at file scope.
- **enum** creates named integer constants grouped under one name. It is useful for related integer values such as categories. It holds whole numbers only, so it is not suitable for a rate such as Rs 2.25 per unit.
- **Naming convention:** constants are written in upper case with underscores. The unit (PAISE or UNITS) is part of the name, so a reader knows what the value means.

Money is stored in paise. The fixed charge uses the L suffix (5000L), so that it is a long constant and matches the long variable that receives it.

Procedure / Algorithm

1. Start with the fixed charge in paise: long paise = FIXED_CHARGE_PAISE.
2. If units are above SLAB2_UNITS (200), add (units − 200) × RATE3_PAISE for the units above the second slab.
3. If units are above SLAB1_UNITS (100), add (the smaller of units and 200, minus 100) × RATE2_PAISE for the units in the second slab.
4. Add (the smaller of units and 100) × RATE1_PAISE for the first slab, which is free.
5. Return paise. The caller divides by 100 to print rupees and divides the remainder to print paise.

Program / Source Code

Program 4: compute_bill() with TODO 1 completed. The macros at the top are unchanged.

```
#include <stdio.h>
```

```
#define SLAB1_UNITS 100            /* first 100 units            */

#define SLAB2_UNITS 200            /* units 101 to 200           */

#define RATE1_PAISE 0              /* first slab is free         */

#define RATE2_PAISE 225            /* Rs 2.25 per unit           */

#define RATE3_PAISE 450            /* Rs 4.50 per unit above 200 */

#define FIXED_CHARGE_PAISE 5000L   /* Rs 50 fixed charge         */
/* Returns the bill in paise, so that no rounding error can occur. */

long compute_bill(int units)

{
    long paise = FIXED_CHARGE_PAISE;

    // TODO 1: Add the energy charge slab by slab using the macros: units above
    //   SLAB2_UNITS at RATE3_PAISE, units from SLAB1_UNITS+1 to SLAB2_UNITS at
    //   RATE2_PAISE, and the rest at RATE1_PAISE
    if (units > SLAB2_UNITS)
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;

    if (units > SLAB1_UNITS) {
        int upper = (units < SLAB2_UNITS) ? units : SLAB2_UNITS;
        paise += (long)(upper - SLAB1_UNITS) * RATE2_PAISE;
    }

    {
        int first = (units < SLAB1_UNITS) ? units : SLAB1_UNITS;
        paise += (long)first * RATE1_PAISE;
    }
    return paise;
}
int main(void)

{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
    }
    return 0;
}
```

Output Screenshot

```
Task 3: compute_bill.c with the original tariff

$ gcc -std=c11 -Wall -Wextra -pedantic task3/compute_bill.c -o task3/compute_bill &&
  ./task3/compute_bill
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

Figure 3.1: Compiling and running compute_bill.c with the original tariff

Text copy of the output with the original tariff:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

```
Task 3: after changing RATE2_PAISE to 230

$ gcc -std=c11 -Wall -Wextra task3/compute_bill_230.c -o task3/compute_bill_230 &&
  ./task3/compute_bill_230
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 165.00
200 units -> Rs 280.00
350 units -> Rs 955.00
```

Figure 3.2: Running compute_bill.c after changing RATE2_PAISE to 230

Text copy of the output with RATE2_PAISE = 230:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 165.00
200 units -> Rs 280.00
350 units -> Rs 955.00
```

Analysis / Discussion

Checking the output by hand

For 150 units, the energy charge is 50 units × 225 paise = 11,250 paise. Adding the fixed charge of 5,000 paise gives 16,250 paise, which is Rs 162.50. For 350 units, the energy charge is 100 × 225 + 150 × 450 = 22,500 + 67,500 = 90,000 paise. Adding the fixed charge gives 95,000 paise, which is Rs 950.00. Both match the expected output of the lab.

Effect of a tariff revision

Changing RATE2_PAISE from 225 to 230 raises the energy charge by 5 paise for each unit in the second slab. For 150 units the bill rises from Rs 162.50 to Rs 165.00 (an increase of Rs 2.50, which is 50 units × 5 paise). For 200 units it rises from Rs 275.00 to Rs 280.00 (100 units × 5 paise). For 350 units it rises from Rs 950.00 to Rs 955.00. Every change agrees with the formula, and only one line of the program was edited. This is the purpose of defining constants in one place.

Why #define is used here

The tariff values are fixed for a billing period and are used only in arithmetic. #define is the simplest choice. It uses no memory, it is replaced at compile time, and it can be used anywhere a constant is needed. The disadvantage is that the compiler does not check the type of a #define, and the name is not available in the debugger.

Alternative concept and complexity analysis: const variables or an enum

If the tariff were written as const variables, for example const long RATE2_PAISE = 225;, the program would give the same output. Each constant would have a type that the compiler checks, and it would be visible in the debugger. A const variable may occupy memory (for example 8 bytes for a long), although a good compiler often removes it. In C, a const variable cannot be used as an array size at file scope. An enum gives integer constants that can be used in case labels, but it cannot store a rate such as 2.25 rupees, so paise values would still be needed. The running time is not affected by the choice. Each bill is still O(1), because each slab is one multiplication and one addition.

Result

Task 3 is complete. All five expected outputs match the lab output. The tariff is defined once, and when RATE2_PAISE is changed to 230, every affected bill changes exactly as the formula predicts.

TASK 4: Use Macros Safely

Task / Problem Statement

Write macros for small repeated calculations, such as converting rupees to paise and finding the larger of two values. Show how the preprocessor expands each macro using gcc -E. Explain why SQUARE(x) written as x*x gives a wrong result for SQUARE(a+1), and how parentheses fix it. Explain why MAX(a, b) can evaluate an argument twice, for example MAX(i++, j). Explain when a function is a better choice than a macro.

Aim

To write safe, fully parenthesised macros, and to demonstrate with gcc -E the pitfalls of macros that are written carelessly.

Concept / Theory

- A macro is replaced by text before compilation. The macro has no type checking and no function call.
- Each parameter and the whole expression must be surrounded by parentheses, for example ((x)*(x)). Without them, the operator precedence of the surrounding code can change the result.
- If an argument has a side effect, such as i++, the argument is copied into the macro and is evaluated each time it appears. This can change the value of a variable more than once.
- gcc -E stops after the preprocessing stage. Running gcc -E -P file.c shows the text that the compiler actually receives, so the expansion of each macro can be checked.

Procedure / Algorithm

1. Define SQUARE_BAD(x) as x*x, and define SQUARE(x) as ((x)*(x)).
2. Call both macros with a+1, where a is 3. The correct answer is 16.
3. Define MAX(a, b) as ((a) > (b) ? (a) : (b)).
4. Call MAX(i++, j) with i = 5 and j = 2. Print the result and the final value of i.
5. Define RUPEES_TO_PAISE(r) as ((long)(r) * 100L) and test it with 162.
6. Run gcc -E -P on the file and read the expanded lines.

Program / Source Code

Program 5: macros and their pitfalls.

```
#include <stdio.h>

#define SQUARE_BAD(x)      x*x                /* no parentheses: wrong    */
#define SQUARE(x)          ((x)*(x))          /* fully parenthesised      */
#define MAX(a, b)          ((a) > (b) ? (a) : (b))
#define RUPEES_TO_PAISE(r) ((long)(r) * 100L)

int main(void)
{
    int a = 3;
```

```
    printf("SQUARE_BAD(a+1) = %d   (expected 16)\n", SQUARE_BAD(a+1));
    printf("SQUARE(a+1)     = %d   (expected 16)\n", SQUARE(a+1));

    int i = 5, j = 2;
    int m = MAX(i++, j);
    printf("MAX(i++, j) = %d, i after the call = %d (i was 5)\n", m, i);

    printf("RUPEES_TO_PAISE(162) = %ld paise\n", RUPEES_TO_PAISE(162));
    return 0;
}
```

Output Screenshot

```
Task 4: macros.c

$ gcc -std=c11 -Wall -Wextra -pedantic task4/macros.c -o task4/macros && ./task4/macros
SQUARE_BAD(a+1) = 7   (expected 16)
SQUARE(a+1)     = 16   (expected 16)
MAX(i++, j) = 6, i after the call = 7 (i was 5)
RUPEES_TO_PAISE(162) = 16200 paise
```

Figure 4.1: Compiling and running macros.c

Text copy of the Program 5 output:

```
SQUARE_BAD(a+1) = 7   (expected 16)
SQUARE(a+1)     = 16   (expected 16)
MAX(i++, j) = 6, i after the call = 7 (i was 5)
RUPEES_TO_PAISE(162) = 16200 paise
```

```
Task 4: gcc -E expansion of the macro lines

$ gcc -E -P task4/macros.c | grep -E 'printf\("(SQUARE|MAX|RUPEES)'
    printf("SQUARE_BAD(a+1) = %d   (expected 16)\n", a+1*a+1);
    printf("SQUARE(a+1)     = %d   (expected 16)\n", ((a+1)*(a+1)));
    printf("MAX(i++, j) = %d, i after the call = %d (i was 5)\n", m, i);
    printf("RUPEES_TO_PAISE(162) = %ld paise\n", ((long)(162) * 100L));
```

Figure 4.2: Macro expansion shown by gcc -E -P

Text copy of the gcc -E -P lines that use the macros:

```
printf("SQUARE_BAD(a+1) = %d   (expected 16)\n", a+1*a+1);
printf("SQUARE(a+1)     = %d   (expected 16)\n", ((a+1)*(a+1)));
printf("MAX(i++, j) = %d, i after the call = %d (i was 5)\n", m, i);
printf("RUPEES_TO_PAISE(162) = %ld paise\n", ((long)(162) * 100L));
```

Analysis / Discussion

Why SQUARE(a+1) gives the wrong result

SQUARE_BAD(a+1) expands to a+1*a+1. Multiplication is evaluated before addition, so the expression is a + (1 × a) + 1 = 3 + 3 + 1 = 7. The fully parenthesised SQUARE(a+1) expands to ((a+1)*(a+1)) = 4 × 4 = 16. The parentheses around x force the whole argument to be evaluated first.

Why MAX(i++, j) evaluates i++ twice

MAX(i++, j) expands to ((i++) > (j) ? (i++) : (j)). With i = 5 and j = 2, the comparison 5 > 2 is true, so the macro evaluates (i++) a second time. The second evaluation returns 6 and makes i equal to 7. The result is 6, when the correct maximum is 5, and i has increased by 2 instead of 1. A function avoids this problem, because the argument is evaluated once before the function is called.

When a function is a better choice than a macro

A function is better when an argument has a side effect, when the type must be checked, when the code must be easy to debug, or when the operation is more than a single expression. A macro is acceptable for very small expressions that have no side effects and that must work with many types.

Alternative concept and complexity analysis: static inline functions

If MAX and SQUARE were written as static inline functions, for example static inline int max_int(int a, int b) { return a > b ? a : b; }, each argument would be evaluated once, and the types would be checked. In time, a macro is expanded with no call at all. A function call adds a few instructions for the jump and the return. In practice, an optimising compiler usually inlines such a small function, so both forms run in O(1) time, and the difference is negligible. In space, a macro is copied at each place it is used. If the macro is used k times, the code contains k copies of its expression. A static inline function has one definition in the source, and the compiler decides how many copies to produce. The space cost of a macro therefore grows with the number of uses, which is O(k) copies of the body.

Result

Task 4 is complete. The macros are fully parenthesised, and the expansions were checked with gcc -E. The output shows the pitfalls: SQUARE_BAD(a+1) gives 7 instead of 16, and MAX(i++, j) evaluates i++ twice. The correctly parenthesised SQUARE gives 16, and RUPEES_TO_PAISE gives 16200 paise for Rs 162.

Design Challenge: Smart Meters and Time-of-Day Tariff

The co-operative plans to install smart meters that report every 15 minutes, and to introduce a time-of-day tariff. The original design should be re-evaluated and kept, modified or redesigned.

Scale of the new data

A meter that reports every 15 minutes produces 4 readings an hour, or 96 readings a day. For 800,000 houses, this is 76.8 million readings a day. If each reading is stored as a 4-byte value, the raw data is about 307 MB per day. For one billing period of 30 days, a single house has about 2,880 readings.

Decision: modify the design, do not redesign it

The original design should be **kept in part and modified**. The following parts remain valid: money in integer paise, the unsigned consumption calculation that handles rollover, and the tariff constants defined in one place. The changes needed are:

- Store the cumulative register and the daily and monthly sums in 32-bit unsigned values, so that long totals cannot overflow a 16-bit type.
- Compute the units for each 15-minute interval with consumption(), then add the interval units to the correct time-of-day band.
- Replace the single set of slab rates with a table of constants, one rate for each time-of-day band, defined once and used by every bill.
- Keep the bill in paise, so the same exact arithmetic is used across all bands.

Effect on time and space complexity

In the original design, one bill takes O(1) time. In the modified design, one bill takes O(m) time, where m is the number of intervals in the billing period, about 2,880 for a month. The whole district of 800,000 houses needs about 2.3 billion interval calculations for one month, so the run is O(N × m), where N is the number of houses. This work can be split into batches, but it is far more than the original design needs. A full redesign would only be needed if the data had to be processed in real time, for example with a streaming database, which is outside the scope of this program.

Test Cases / Validation Table

The table records different inputs, expected results, actual results and whether each test passed. Rows marked "Pitfall shown" or "Fail" are deliberate. They demonstrate the problem that the chosen design avoids.

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1	Task 2: consumption(1200, 1530), normal reading	330 units	330 units	Pass
2	Task 2: consumption(65400, 120), rollover	256 units	256 units	Pass
3	Task 2: consumption(65535, 0), rollover by one unit	1 unit	1 unit	Pass
4	Task 2: consumption(500, 500), no change	0 units	0 units	Pass
5	Task 2: consumption(0, 65535), full range	65535 units	65535 units	Pass
6	Task 3: 0 units	Rs 50.00	Rs 50.00	Pass
7	Task 3: 100 units (end of slab 1)	Rs 50.00	Rs 50.00	Pass
8	Task 3: 101 units (first chargeable unit)	Rs 52.25	Rs 52.25	Pass
9	Task 3: 150 units	Rs 162.50	Rs 162.50	Pass
10	Task 3: 200 units (end of slab 2)	Rs 275.00	Rs 275.00	Pass
11	Task 3: 201 units (first unit at Rs 4.50)	Rs 279.50	Rs 279.50	Pass
12	Task 3: 350 units	Rs 950.00	Rs 950.00	Pass
13	Task 3: RATE2_PAISE changed to 230, 150 units	Rs 165.00	Rs 165.00	Pass
14	Task 4: SQUARE_BAD(a+1), a = 3	16	7	Pitfall shown
15	Task 4: SQUARE(a+1), a = 3	16	16	Pass
16	Task 4: MAX(i++, j), i = 5, j = 2	Maximum 5, i becomes 6	Result 6, i becomes 7	Pitfall shown
17	Task 4: RUPEES_TO_PAISE(162)	16200 paise	16200 paise	Pass
18	Float total of 4,000 bills of Rs 27.30	Rs 109200.00	Rs 109196.33	Fail (float error shown)

Test No.	Input / Scenario	Expected Output	Actual Output	Status
19	Paise total of the same 4,000 bills	Rs 109200.00	Rs 109200.00	Pass

Final Submission Checklist

☒ Student and course details completed. The Submission Date field is left blank because the lab page does not give it.

☒ Task statement, aim, theory and algorithm included for every task.

☒ Program code is readable and compiled without warnings using gcc -Wall -Wextra -pedantic.

☐ Output screenshots included. The highlighted placeholders must be replaced with your own screenshots.

☐ Analysis and result reviewed and written in your own words.

☒ Test cases completed for each applicable task.

☐ GitHub repository link checked. The repository https://github.com/mukeshtharun/CSA02-1971260006D-Assignment1 must exist, with one folder per task, a README.md and at least one commit per task.